

GOLANG

Living Textbook cho Software Engineering và DevOps/SRE

Edition nền móng

Được kiểm chứng với Go 1.27.1 - 20-09-2026

Markdown là nguồn gốc. PDF là bản đọc được, có thể tái tạo cục bộ.

Mục lục của edition này

Đây là edition nền móng. Mục lục tổng thể và thứ tự các phần tiếp theo được giữ trong `book/README.md` để có thể mở rộng mà không giả vờ rằng chúng đã được viết xong.

- Trước khi viết dòng Go đầu tiên
- Mở cửa vào Go
- Chương 1 — Đọc và viết một chương trình Go
- Chương 2 — Khi một bản sao vẫn chia sẻ dữ liệu

Trước khi viết dòng Go đầu tiên

Go không bắt đầu bằng câu hỏi “cú pháp nào còn thiếu?”. Nó bắt đầu bằng một vấn đề kỹ thuật rất cụ thể: đến năm 2007, máy tính đã nhanh hơn rất nhiều, nhưng việc xây và sửa một hệ thống phần mềm lớn vẫn bị kéo chậm bởi ngôn ngữ, build system và lượng bookkeeping mà lập trình viên phải mang theo.

Ở Google khi ấy, Robert Griesemer, Rob Pike và Ken Thompson làm việc giữa những codebase rất lớn, hệ thống nối mạng và phần cứng đang chuyển mạnh sang nhiều lõi. C++ và Java là lựa chọn phổ biến cho server; chúng đem lại những năng lực quan trọng, đồng thời tạo ra một lượng phức tạp đáng kể trong quá trình build, quản lý dependency và tổ chức chương trình. Ở phía khác, các ngôn ngữ động cho cảm giác phát triển nhanh hơn, nhưng đánh đổi một phần kiểm tra kiểu và đặc tính thực thi. Bài toán không phải là chọn một phe thắng phe còn lại. Bài toán là liệu một ngôn ngữ có thể đem hiệu quả biên dịch, hiệu năng và an toàn kiểu vào cùng một quy trình làm việc nhẹ hơn hay không.

Ngày 21 tháng 9 năm 2007, ba người bắt đầu phác thảo mục tiêu của một ngôn ngữ mới trên bảng trắng. Vài ngày sau, ngày 25 tháng 9, Pike đề xuất cái tên Go. Thiết kế ban đầu diễn ra song song với công việc khác; đầu năm 2008, Thompson đã có compiler thử nghiệm sinh mã C để thử các ý tưởng. Đến giữa năm, dự án đủ rõ để hướng tới compiler dùng trong thực tế. Ian Lance Taylor bắt đầu GCC front end vào tháng 5 năm 2008; Russ Cox gia nhập cuối năm ấy và giúp đưa prototype, library cùng tooling tiến gần một ngôn ngữ có thể dùng.



Hình 1 — Dòng thời gian không kể mọi release. Nó chỉ giữ các bước làm thay đổi lời hứa của Go với người viết phần mềm: công khai, ổn định, triển khai lại nền tảng, dependency rõ ràng và generics tương thích ngược.

Go được mở mã nguồn ngày 10 tháng 11 năm 2009. Ba năm đầu là giai đoạn ngôn ngữ còn thay đổi nhanh: nhóm thiết kế và cộng đồng thử, bỏ, sửa, rồi mới chốt một nền tảng ổn định. Go 1 phát hành ngày 28 tháng 3 năm 2012. Từ đây, Go 1 Compatibility Promise trở thành một phần của văn hóa dự án: code nguồn viết cho Go 1 được kỳ vọng tiếp tục build và chạy trong các bản Go 1 sau, với các ngoại lệ được nêu rõ. Lời hứa ấy làm thay đổi cách một feature được cân nhắc. Một bổ sung không chỉ cần hữu ích hôm nay; nó phải sống được cùng hệ sinh thái trong nhiều năm.

Go 1.5 là một cột mốc về implementation. Toolchain compiler chuyển từ C sang Go, runtime cũng trở thành Go kèm một phần assembly. Garbage collector được thiết kế lại để chạy phần lớn công việc đồng thời với chương trình; scheduler cho phép giá trị mặc định của GOMAXPROCS chuyển từ 1 sang số logical CPU. Đó là những thay đổi nằm dưới mặt ngôn ngữ, nhưng

chúng làm rõ mục tiêu ban đầu: một chương trình Go cần hợp với phần cứng nhiều lỗi, không bắt người viết phải tự gánh mọi chi tiết của runtime.

Những thay đổi lớn sau đó cũng đi theo nhịp tiến hóa thay vì đập đi làm lại. Go 1.11 đưa modules vào như một lựa chọn thay cho GOPATH, với versioning và package distribution tích hợp vào toolchain; mô hình này dần thành cách quản lý dependency quen thuộc. Go 1.18, phát hành năm 2022, đưa type parameters vào ngôn ngữ. Đến Go 1.27 năm 2026, generic methods nổi phần còn thiếu vào câu chuyện đó: method declaration có thể có type parameters riêng. “Go 2” vì thế không thành một nhánh rewrite tách rời; nhiều ý tưởng được thử, thảo luận và đưa dần vào dòng Go 1.x khi chúng minh được giá trị của chúng.

Một dòng họ ý tưởng, không phải một cây sao chép

Go thuộc nhiều dòng tư tưởng cùng lúc. Cú pháp cơ bản đặt nó gần họ C. Cách nhìn về declaration, package và sự rõ ràng cấu trúc nhận ảnh hưởng đáng kể từ Pascal, Modula và Oberon. Concurrency có một lịch sử dài hơn: CSP của Tony Hoare đi qua Newsqueak, Alef và Limbo trước khi channels trở thành một phần của Go. Những đường này giải thích vì sao một số ý tưởng nghe quen, chứ không làm Go thành bản sao của bất kỳ ngôn ngữ nào trong số đó.



Hình 2 — Nhánh CSP giải thích lịch sử của channels; nhánh C và Pascal/Modula/Oberon giải thích những điểm xuất phát khác. Thiết kế cuối cùng vẫn được tạo cho bài toán engineering mà Go nhắm đến.

Một ngôn ngữ được sinh ra cho codebase lớn không chỉ cần syntax gọn. Nó cần một cách làm việc chung quanh syntax: build nhanh, formatter, package/dependency rõ, compiler cho phản hồi sớm và tooling có thể phân

tích source. Đó là lý do Go thường được nói đến cùng `gofmt`, `go test` và `go command`, thay vì chỉ cùng một danh sách tính năng ngôn ngữ.

Thiết kế bằng cách từ chối

Trong giai đoạn đầu, nhóm sáng lập đặt nặng đồng thuận: một feature chỉ đi tiếp khi cả nhóm thấy nó thực sự cần. Câu hỏi không dừng ở “có thể thêm gì?”, mà còn là “có thể bỏ gì để hệ thống vẫn đủ sức làm việc?”. Kết quả là một bộ trade-off có chủ ý.

Một điều đáng nhớ: sự vắng mặt của inheritance hierarchy, operator overloading, exception-style error handling hay generics trong giai đoạn đầu không phải bản tuyên ngôn rằng các cơ chế ấy luôn tệ. Mỗi cơ chế có ích trong những ngữ cảnh khác. Go ưu tiên một mô hình dễ đọc, dễ phân tích, biên dịch nhanh và dễ giữ ổn định; cái giá là một số biểu đạt quen thuộc không có mặt hoặc đến muộn hơn.

Không có type hierarchy buộc người đọc đi tìm cây kế thừa; thay vào đó, interface và composition sẽ được nói kỹ khi ta đã đọc vững code cơ bản. Không có operator overloading giúp ý nghĩa của các toán tử ít thay đổi theo type. Error được trả như value thay vì theo đường exception-style. Và generics được hoãn lại cho đến khi thiết kế có thể đem giá trị đủ lớn so với độ phức tạp nó thêm vào type system và toolchain. Đây là các lựa chọn kỹ thuật có thể tranh luận, không phải luật đạo đức.

Nhìn lịch sử theo cách này giúp giải thích cảm giác đầu tiên khi đọc Go. Bạn sẽ thấy nhiều câu ngắn, ít keyword, declaration rõ và tooling hiện diện sớm. Những đặc điểm ấy là dấu vết của một mục tiêu: giảm ma sát cho người đang cùng người khác xây, đọc và sửa phần mềm lâu dài.

Sau gần hai thập kỷ tiến hóa, cách tốt nhất để gặp các quyết định ấy không phải đọc thêm lịch sử. Hãy đặt chúng trước mắt trong một tệp `.go` rất nhỏ.

Ghi chú kiểm chứng

1. Go Team. Frequently Asked Questions: Origins and Design. go.dev/doc/faq
2. Cox, Russ. Toward Go 2. 13 July 2017. go.dev/blog/toward-go2

3. Go Team. Go 1 and the Future of Go Programs. March 2012. go.dev/doc/go1compat
4. Gerrand, Andrew. Go 1.5 is released. 19 August 2015. go.dev/blog/go1.5
5. Bonventre, Andrew. Go 1.11 is released. 24 August 2018. go.dev/blog/go1.11
6. Go Team. Go 1.18 is released! 15 March 2022. go.dev/blog/go1.18
7. Go Team. Go 1.27 Release Notes. 2026. go.dev/doc/go1.27

Mở cửa vào Go

Bạn không cần thuộc ngay danh sách keyword để bắt đầu. Điều cần trước tiên là nhìn một tệp Go và biết: phần nào đặt tên cho chương trình, phần nào tạo giá trị, phần nào quyết định đường đi, và phần nào chỉ là một lời gọi hàm.

Cuốn sách này không đi bằng các mẫu bài học quá ngắn. Nó đi bằng những lần thay đổi cách nhìn: đọc được code trước, nhìn được giá trị chạy qua code sau, rồi mới đến những lúc nhiều goroutine, mạng, dữ liệu và production khiến trực giác ban đầu không còn đủ. Khi một chủ đề cần một lỗi, một trace hay một thí nghiệm để thấy rõ, chương đó sẽ bắt đầu từ bằng chứng ấy.

Cách đọc: đừng lướt qua code block. Mỗi lần gặp một đoạn ngắn, hãy thử nói thành lời nó tạo giá trị gì, giá trị đó được đặt tên ở đâu và điều gì sẽ xảy ra nếu đổi một token. Compiler là phản hồi nhanh nhất cho dự đoán của bạn.

Chương đầu không có mục tiêu “biết hết Go”. Mục tiêu nhỏ hơn, nhưng thiết yếu: bạn có thể đọc một chương trình console ngắn từ trên xuống, tự sửa nó, và hiểu vì sao một thay đổi là hợp lệ hoặc không hợp lệ.

Chương 1 — Đọc và viết một chương trình Go

Một tệp nguồn có hình học riêng: phần đầu đặt nó vào package, vài declaration đứng ở cấp tệp, rồi các block lồng vào nhau. Ta sẽ bắt đầu từ một chương trình rất nhỏ: nó nhận một mã trạng thái và in ra một nhãn mà người trực vận hành có thể đọc. Ví dụ đủ thực để có dữ liệu, lựa chọn và thông điệp, nhưng chưa che mất ngôn ngữ dưới một dự án.

```
package main

import "fmt"

const service = "checkout"

func classify(status int) string {
    if status >= 500 {
        return "failed"
    }
    return "healthy"
}

func main() {
    status := 503
    label := classify(status)
    fmt.Println(service, label)
}
```



Hình 3 — Hãy nhìn figure này như bản đồ định vị trước khi đọc chi tiết. Nó cho thấy package name, main package, function declaration và entry function là các khái niệm có quan hệ, nhưng không phải là một khái niệm duy nhất.

Chưa cần đọc từng dòng. Hãy chỉ theo ba câu hỏi: chương trình bắt đầu ở đâu, giá trị nào được tạo, và giá trị nào bị gửi ra màn hình? Câu trả lời là main, literal 503 cùng các giá trị trả về từ classify, và lời gọi Println.

Bản đồ của một tệp nguồn

Từ đầu xuống cuối, tệp trên có bốn vùng: package nói tệp thuộc chương trình nào; import cho phép dùng tên từ package khác; khai báo ở cấp tệp tạo const và hàm; cuối cùng, thân hàm main là nơi một chuỗi câu lệnh được chạy.

Dòng hoặc mảnh code	Nó là gì khi đọc	Nó có tác dụng gì
<code>package main</code>	package declaration	Đặt tệp vào package có tên main. Một executable hoàn chỉnh cần main package và func main không argument, không result.
<code>import "fmt"</code>	import declaration	Cho phép tham chiếu fmt từ standard library.
<code>const service = "checkout"</code>	declaration + literal	Gắn một tên không đổi với chuỗi.
<code>func classify(status int) string</code>	function declaration + signature	Đặt tên hàm, tham số kiểu int và kết quả kiểu string.
<code>status >= 500</code>	expression	Tính ra một giá trị bool.
<code>return "failed"</code>	statement	Kết thúc lời gọi hiện tại với một string.
<code>status := 503</code>	short variable declaration	Tạo biến status trong block main.
<code>fmt.Println(...)</code>	function call statement	Gọi hàm để có hiệu ứng in ra stdout.

Các nhãn này cho ta một cách chỉ vị trí khi compiler báo lỗi: lỗi có thể nằm trong một biểu thức, kiểu của một lời gọi, hay phạm vi của một tên thay vì chỉ là cảm giác mơ hồ rằng “code bị sai”.



Hình 4 — Đây là mô hình đọc, không phải sơ đồ nội bộ chính xác của Go compiler. Nó giải thích vì sao một dấu ngoặc, một tên không tồn tại và phép cộng sai kiểu thường bị bắt trước khi chương trình có cơ hội chạy.

Một tên chỉ có nghĩa trong phạm vi của nó

Ở dòng `const service`, `service` là identifier: một cái tên do người viết chọn. `fmt`, `Println`, `classify`, `status` và `label` cũng là identifier. Ngược lại, `package`, `import`, `const`, `func`, `if` và `return` là keyword; chúng dành riêng cho cấu trúc ngôn ngữ nên bạn không thể dùng chúng để tự đặt tên.

Tên không tự mang giá trị. Câu `status := 503` tạo một biến, rồi gán giá trị literal 503 vào biến đó. Trong cùng block, câu `label := classify(status)` tạo một biến khác, nhận kết quả mà hàm trả về. Tên tồn tại từ điểm khai báo đến cuối block bao quanh.

Phóng to một statement

Compiler không đọc `status := 503` như một câu tiếng Anh. Nó tách câu này thành các token có vai trò cú pháp, rồi ghép các token thành statement.

<code>status</code>	<code>:=</code>	<code>503</code>
identifier	operator	integer literal

Đây là một mức đọc khác với mức file hoặc function: identifier gọi lại một tên trong source, `:=` là operator khai báo-ngắn, còn `503` là integer literal. Khi nhìn code theo từng mức như vậy, một compiler message có chỗ để bám vào.

```
func main() {  
    status := 503  
    if status >= 500 {  
        label := "failed"  
        fmt.Println(label)  
    }  
    // fmt.Println(label) // không hợp lệ: label đã hết scope  
}
```

Khối `if` được tạo bởi cặp ngoặc nhọn. `label` chỉ hữu ích bên trong khối đó. Đây là một giới hạn có chủ ý: người đọc không phải lục lại toàn bộ tệp để đoán một tên có còn sống hay không.



Hình 5 — Scope không mô tả dữ liệu được lưu ở đâu trong bộ nhớ. Nó chỉ giới hạn nơi tên được phép được tham chiếu trong source.

Dừng lại để dự đoán: nếu bỏ comment ở dòng cuối, compiler phàn nàn về giá trị hay về tên? Câu trả lời là tên: `label` không còn được khai báo trong scope mà lời gọi `Println` đang đứng.

Làm chương trình lớn lên, mỗi lần một ý

Đừng viết đầy đủ từ đầu. Ta quan sát cùng một ý định khi nó nhận thêm một yêu cầu. Mỗi phiên bản giữ lại phần cũ có ích và chỉ thêm một khái niệm.

Literal: một kết quả cố định

```
package main

import "fmt"

func main() {
    fmt.Println("checkout healthy")
}
```

"checkout healthy" là string literal. Chương trình có một statement gọi hàm và không có quyết định nào. Nó đúng nếu thông điệp không bao giờ thay đổi.

Variable: dữ liệu thay đổi được đặt tên

```
func main() {
    service := "checkout"
    label := "healthy"
    fmt.Println(service, label)
}
```

Hãy nhìn một thay đổi trước khi học rule. Lần đầu, ta tạo status và cho nó giá trị. Lần sau, status đã có trong cùng block; ta chỉ thay giá trị của nó.

```
status := 503
status = 200
```

Vì dòng thứ hai không tạo một tên mới, nó dùng = thay vì :=. Short declaration chỉ dùng trong function body. Với nhiều tên ở vế trái, tên cũ có thể xuất hiện lại khi chúng đã có trong cùng block (hoặc parameter list tương ứng), type không đổi, và có ít nhất một non-blank variable mới. _ không được tính là tên mới. Đây là trường hợp tiện lợi khi bổ sung một kết quả mới mà vẫn giữ một biến đã có.

Muốn làm gì?	Viết gì?	Điều compiler cần biết
Tạo một biến mới và suy ra kiểu	<code>name := value</code>	Ít nhất một tên ở vế trái là mới trong scope hiện tại.
Gán lại biến đã có	<code>name = value</code>	Tên đã tồn tại và giá trị mới phù hợp kiểu.
Đặt giá trị không đổi ở cấp tệp	<code>const name = value</code>	Giá trị phải là hằng; không dùng <code>:=</code> .
Khai báo rõ kiểu trước	<code>var name Type = value</code>	Dùng khi kiểu hoặc zero value cần được nói rõ.

Expression: để chương trình tự kết luận

Literal 503 không tự nói rằng service thất bại. Ta cần một biểu thức tạo ra quyết định. `status >= 500` đọc là “status lớn hơn hoặc bằng 500”, và kết quả của nó là bool: true hoặc false.

```
func main() {
    status := 503
    if status >= 500 {
        fmt.Println("checkout failed")
    }
}
```

if nhận một biểu thức bool. Khi biểu thức là true, block của if được thực hiện; khi false, nó bị bỏ qua. Dấu ngoặc không chỉ để trang trí: chúng xác định chính xác nhóm câu lệnh phụ thuộc vào điều kiện.

Toán tử có thứ tự ưu tiên. Viết ngoặc khi chúng làm ý định rõ hơn thay vì bắt người đọc nhớ luật ưu tiên:

```
urgent := status >= 500 ||
    (status >= 400 && service == "checkout")
```

`&&` nghĩa là cả hai vế phải đúng; `||` nghĩa là ít nhất một vế đúng. Biểu thức bên trong ngoặc chạy trước phần `||`. Không cần biến mọi phép toán thành một câu đố — một tên như `urgent` thường dễ đọc hơn một điều kiện bị nén.

Function: đặt tên cho một quyết định có thể tái dùng

Khi điều kiện có ý nghĩa riêng, đưa nó vào hàm. Người đọc khi đó gặp một tên ở đúng mức trừu tượng thay vì phải giải mã lại cùng một điều kiện ở mọi nơi.

```
func classify(status int) string {  
    if status >= 500 {  
        return "failed"  
    }  
    return "healthy"  
}
```

Signature nói rõ hợp đồng tối thiểu: `classify` nhận một `int` và trả về một `string`. `status` là parameter, chỉ tồn tại trong lời gọi hàm. `return` không in gì cả; nó gửi một giá trị về nơi gọi. Vì thế câu `label := classify(status)` đọc theo thứ tự này: đọc `status` hiện có, gọi `classify` với giá trị đó, nhận `string` kết quả, rồi tạo `label`.

Cùng một quyết định, hai cách tổ chức

Nếu có nhiều trạng thái rời rạc, `switch` thường bộc lộ ý định tốt hơn một chuỗi `if`. Nó không “mạnh hơn”; nó chỉ đặt các nhánh cùng cấp cạnh nhau.

Đầu vào	Dạng phù hợp	Lý do đọc dễ hơn
Một ngưỡng: <code>status >= 500</code>	<code>if</code>	Điều kiện là một mệnh đề đúng/sai.
Vài giá trị cụ thể: 200, 404, 503	<code>switch</code>	Mỗi case là một lựa chọn cùng cấp.
Một thao tác lặp trên danh sách	<code>for</code> hoặc <code>range</code>	Ý định là làm lại, không phải chọn một nhánh.

```
func severity(status int) string {
    switch {
    case status >= 500:
        return "critical"
    case status >= 400:
        return "warning"
    default:
        return "normal"
    }
}
```

switch trên không có expression sau từ switch, nên từng case là bool expression. Go chạy case đầu tiên đúng rồi rời switch; không có fallthrough ngầm như một số ngôn ngữ khác. Thứ tự ở đây là dữ liệu của thiết kế: nếu đưa status >= 400 lên trước, 503 sẽ bị phân loại warning. Một compiler không thể bắt lỗi ý nghĩa đó cho bạn; đó là việc của người đọc và test.

Lặp là đi qua một tập giá trị

Một for có thể diễn tả vòng lặp điều kiện, nhưng range thường trực tiếp hơn khi ta muốn đi qua các phần tử của một collection.

```
statuses := []int{200, 503, 404}

for _, status := range statuses {
    fmt.Println(status, severity(status))
}
```

[]int là slice của int; cặp ngoặc nhọn tạo literal gồm ba phần tử. range cung cấp index và value. Tên _ bảo Go bỏ index đi vì ta không cần nó. Mỗi lượt, status là giá trị hiện tại; lời gọi severity dùng nó và Println in kết quả. Ta sẽ quay lại slice, bộ nhớ và aliasing ở chương sau — hiện tại chỉ cần thấy luồng đọc của vòng lặp.

Kiểu giới hạn những phép có nghĩa

Go suy ra nhiều kiểu từ literal, nhưng vẫn kiểm tra chúng chặt. `int` biểu diễn số nguyên; `bool` biểu diễn `true` hoặc `false`; `string` là một sequence of bytes không thể sửa trực tiếp. Text trong Go thường dùng UTF-8, và range trên `string` có semantics liên quan UTF-8/rune, nhưng `string` vẫn có thể chứa chuỗi byte không phải UTF-8 hợp lệ. Một biến có kiểu xác định; phép toán đòi các vế tương thích.

```
status := 503
message := "status: " + status // không hợp lệ
```

Toán tử `+` ở đây có thể nối hai `string` hoặc cộng hai số, nhưng không tự đoán cách trộn `string` và `int`. Chuyển đổi là một quyết định phải viết ra:

```
message := "status: " + fmt.Sprint(status)
```

`fmt.Sprint` tạo `string` để dùng cho hiển thị. Trong những ranh giới nghiêm túc hơn, ta sẽ phân biệt conversion với parsing và kiểm tra lỗi. Bây giờ điểm quan trọng là: kiểu giúp phát hiện một phép ghép vô nghĩa ngay tại nơi bạn viết nó.

Đọc compiler như một người cộng tác khó tính

Phần này là một cuộc điều tra nhỏ, không phải checklist lệnh. Trong lab `part1-reading-go`, giữ một bản đang chạy được. Sau đó tạo từng lỗi một, dự đoán trước, chạy compiler, và phân loại thông báo. Đừng sửa ngay khi vừa thấy chữ đỏ.

Case	Thay đổi có chủ ý	Dự đoán cần kiểm tra
A	Viết <code>status = 503</code> khi <code>status</code> chưa tồn tại	Lỗi thuộc về identifier hay giá trị?
B	Đổi <code>classify</code> nhận <code>string</code> nhưng caller vẫn đưa <code>status</code> kiểu <code>int</code>	Compiler nói về function hay type của argument?
C	Dùng <code>label</code> sau khi rời <code>if</code> block	Tên biến mất vì giá trị bị xóa hay vì scope kết thúc?

Kết quả dưới đây được chạy bằng Go 1.27.1 trên ba source file tối thiểu, mỗi case chỉ có một lỗi chủ ý. Dòng dài ở Case B được ngắt khi dàn trang; nội dung của compiler message được giữ nguyên.

```
[case-a]
# command-line-arguments
.\main.go:4:2: undefined: status

[case-b]
# command-line-arguments
.\main.go:9:15: cannot use status (variable of type int)
    as string value in argument to classify

[case-c]
# command-line-arguments
.\main.go:8:6: undefined: label
```

Case A là lỗi tên: compiler không thể gán cho một identifier chưa được khai báo. Case B đi xa hơn, vì tên tồn tại nhưng type của giá trị đưa vào không khớp parameter. Case C quay lại scope: `label` từng tồn tại trong block con, nhưng không thể được tham chiếu từ câu lệnh đứng sau block đó. Thông báo có thể khác đôi chút giữa phiên bản Go, còn ba loại lỗi thì vẫn là ba loại lỗi bạn cần nhìn ra.

Bài đọc cuối chương: không cần viết chương trình mới. Hãy nhìn hàm `severity`, dự đoán kết quả cho 200, 404 và 503, rồi viết một test table nhỏ cho ba dự đoán đó. Nếu test thất bại, lần theo branch được chọn thay vì sửa output một cách mù.

Đến đây, bạn đã có từ vựng để đọc phần lớn tệp Go nhỏ: `package` đưa tệp vào một đơn vị, `declaration` đặt tên, `expression` tạo giá trị, `statement` điều phối hiệu ứng, `block` giới hạn scope, và `function` biến một đoạn quyết định thành đơn vị có hợp đồng. Chương tiếp theo sẽ mở bằng một hiện tượng khó chịu hơn: hai biến trông độc lập, nhưng một thay đổi qua biến này lại hiện ra ở biến kia. Câu hỏi là chúng đã chia sẻ điều gì.

Chương 2 — Khi một bản sao vẫn chia sẻ dữ liệu

Chạy đoạn này trước khi cố giải thích nó. Nếu bạn đoán a và b khác nhau sau dòng `b[0] = 99`, đó là một dự đoán rất hợp lý - và cũng là cánh cửa vào phần này.

```
a := []int{10, 20, 30}
b := a
b[0] = 99

fmt.Println(a)
fmt.Println(b)
```

Kết quả từ lab part2-values là:

```
[99 20 30]
[99 20 30]
```

Đừng ghi nhớ riêng một câu “slice nguy hiểm”. Thay vào đó, ta sẽ lần ngược từ kết quả: Go đã copy cái gì khi chạy `b := a`, và dữ liệu nào không được copy?

Một baseline để dự đoán

Với một `int`, assignment tạo ra một giá trị độc lập. Cả a và b đều có giá trị riêng của chính chúng.

```
a := 10
b := a
b = 99

fmt.Println(a, b) // 10 99
```

Go đã copy value của a vào b. Sau đó b nhận value mới; a không có lý do gì để đổi. Đây là quy tắc nền: assignment copy value. Điều còn thiếu là một value có thể chứa gì.

Array giúp ta kiểm tra quy tắc ấy với nhiều phần tử. Độ dài là một phần của type, nên `[3]int` và `[4]int` là hai array type khác nhau. Khi array được gán, toàn bộ array value được copy.

```
a := [3]int{10, 20, 30}
b := a
b[0] = 99

fmt.Println(a) // [10 20 30]
fmt.Println(b) // [99 20 30]
```



Hình 6 — Assignment của array copy các phần tử. Việc thay `b[0]` chỉ chạm vào array `b`.

Đến đây, trực giác “copy nghĩa là độc lập” vẫn đúng. Sự thay đổi xuất hiện chỉ khi ta bỏ một ký tự trong type: `[3]int` trở thành `[]int`.

Khoảnh khắc slice đổi kết quả

```
a := []int{10, 20, 30}
b := a
b[0] = 99
```

`[]int` không phải array không ghi độ dài. Nó là slice type. Một slice value mô tả một đoạn liên tiếp của underlying array. Ngôn ngữ định nghĩa slice value có length, capacity và một reference tới underlying array. Đó là mental model hữu ích cho semantics; nó không phải lời hứa rằng mọi Go implementation phải dùng đúng một struct hay ABI như hình.



Hình 7 — `b := a` copy slice value, gồm của số nhìn vào dữ liệu, len và cap. Cả hai descriptor vẫn dẫn tới cùng underlying array, nên `b[0]` thay đổi phần tử mà `a` cũng nhìn thấy.

Vì vậy hai câu sau cùng đúng, và không mâu thuẫn:

Câu nói	Đúng ở đâu?
Assignment copy value.	<code>b</code> nhận một slice value riêng: đổi len của <code>b</code> bằng <code>reslice</code> không tự đổi len của <code>a</code> .
Hai slice có thể cùng đổi dữ liệu.	Slice value có thể cùng reference một underlying array; mutation một element hiện ra qua mọi slice cùng nhìn element đó.

Bảng 1 — Copy value và chia sẻ storage là hai tầng khác nhau của cùng một phép gán.

Từ aliasing dùng cho tình huống nhiều đường tham chiếu đến cùng dữ liệu. Nó không phải lỗi tự thân. Aliasing giúp cắt một cửa sổ trên buffer mà không copy cả buffer. Nó thành bug khi một người đọc tưởng mình có dữ liệu riêng nhưng thực tế lại đang sửa vùng dùng chung.

Cửa sổ, length và capacity

Đặt một slice lên một slice khác để thấy descriptor thực sự đang mô tả gì.

```
base := []int{10, 20, 30, 40, 50}
x := base[1:3]

fmt.Println(x)          // [20 30]
fmt.Println(len(x))     // 2
fmt.Println(cap(x))     // 4
```

x bắt đầu ở phần tử 20 của base và kết thúc trước 40. Vì thế length là 2. Capacity đếm số phần tử từ điểm bắt đầu của x đến cuối backing array mà x hiện có thể mở rộng tới; ở đây là 20, 30, 40, 50, nên là 4.



Hình 8 — x nhìn thấy hai phần tử đầu của cửa sổ. cap(x) không nói x đang có bốn phần tử; nó nói x có thể được reslice hoặc append trong phạm vi backing array hiện tại.

Hãy tự dự đoán trước khi chạy:

```
base := []int{1, 2, 3, 4}
a := base[:2]
b := base[1:3]
a[1] = 99

fmt.Println(base)
fmt.Println(a)
fmt.Println(b)
```

Kết quả là [1 99 3 4], [1 99], [99 3]. a[1] và b[0] đều chỉ vào phần tử base[1]. Hình vẽ không thay thế được việc lần index: nó chỉ cho biết vì sao có thể có một phần tử chung để lẫn.

append là một phép thử về capacity

append không hứa luôn dùng lại array cũ, cũng không hứa luôn tạo array mới. Nó trả về resulting slice. Khi backing array còn chỗ, implementation có thể dùng lại; khi capacity không đủ, append phải cấp underlying array mới đủ lớn cho kết quả. Vì return value mang descriptor có thể đã đổi, viết `s = append(s, value)` là dạng thông thường.

```
s := make([]int, 2, 4)
s[0], s[1] = 10, 20
old := s

s = append(s, 30)
s[0] = 99

fmt.Println(old) // [99 20]
fmt.Println(s)   // [99 20 30]
```

Ở đây `s` còn capacity. `old` và `s` có length khác nhau, nhưng vẫn có thể cùng nhìn hai element đầu của backing array. Ngược lại, full slice expression bên dưới giới hạn capacity của view ở 2, nên append buộc phải có storage khác cho grown:

```
old := []int{10, 20}
s := old[:2:2]
grown := append(s, 30)
grown[0] = 99

fmt.Println(old)    // [10 20]
fmt.Println(grown)  // [99 20 30]
```



Hình 9 — Hình mô tả hai semantics mà code phải chịu được. Đừng suy luận allocation từ một capacity tĩnh cở; nếu code cần một kết quả riêng, hãy tạo bản copy có chủ ý.

Một aliasing bug có người dùng thật

Một function “chỉ chuẩn bị preview” đôi khi phá dữ liệu của caller.

```
func redactPreview(fields []string) {  
    fields[0] = "***"  
}  
  
requestFields := []string{"api-key", "region"}  
redactPreview(requestFields)  
fmt.Println(requestFields) // [*** region]
```

Parameter fields nhận một slice value mới, nhưng descriptor ấy vẫn reference storage của requestFields. Việc đổi element là mutation shared storage. Nếu mục tiêu thật sự là tạo preview độc lập, copy phần tử sang một slice mới trước:

```
func redactedCopy(fields []string) []string {
    preview := make([]string, len(fields))
    copy(preview, fields)
    preview[0] = "***"
    return preview
}
```

copy sao chép tối đa số phần tử vừa với destination và trả về số phần tử đã copy. Ở đây make tạo backing array mới, nên preview độc lập ở từng slice elements. Đổi lấy independence là allocation và chi phí copy; với buffer lớn hoặc đường xử lý nóng, đó là một quyết định cần có lý do. Với một preview cần an toàn khỏi mutation, nó diễn đạt đúng intent.

Điều tra ngắn: hãy viết test cho redactedCopy. Test cần chứng minh đồng thời hai điều: kết quả có *** ở phần tử đầu, và requestFields ban đầu vẫn là api-key. Chỉ kiểm tra output preview chưa đủ để bắt aliasing bug.

String: bytes trước, text sau

String cũng là value, nhưng nó có luật riêng: string là immutable sequence of bytes. len đếm byte; index trả về byte. Text thường là UTF-8, song Go không yêu cầu mọi string phải chứa UTF-8 hợp lệ.

```
text := "Việt"
fmt.Println(len(text)) // 6

for index, value := range text {
    fmt.Println(index, value)
}
```

Lab chạy đoạn này với kết quả:

```
6
0 86
1 105
2 7879
5 116
```

V và i mỗi chiếm một byte. ệ được mã hóa bằng ba byte UTF-8, vì vậy rune tiếp theo bắt đầu ở byte index 5. range trên string decode UTF-8: giá trị thứ hai là rune, thường dùng để biểu diễn Unicode code point. byte là alias của uint8; rune là alias của int32. Code point vẫn không đồng nghĩa với “một ký tự người dùng nhìn thấy”: một grapheme có thể gồm nhiều code point. Ta chưa cần giải quyết segmentation ở đây; chỉ cần không nhầm lẫn với số ký tự hiển thị.

String không cho gán trực tiếp `text[0] = 'v'`. Khi cần dữ liệu byte có thể sửa, chuyển sang `[]byte`, thao tác, rồi tạo string mới nếu cần. Việc chuyển đổi đó diễn đạt một thay đổi ý định: từ text immutable sang buffer mutable.

Pointer là câu hỏi khác

Sau slice, pointer dễ bị giải thích sai như “slice nhưng rõ hơn”. Pointer là một value trực tiếp reference một variable khác; slice là một descriptor cho segment của underlying array. Chúng có thể cùng dẫn tới sharing, nhưng chúng trả lời hai câu hỏi khác nhau.

```
count := 10
p := &count
*p = 99
fmt.Println(count) // 99
```

Ở đây `&count` tạo pointer tới variable `count`, còn `*p` truy cập variable đó. Pointer sẽ có chương riêng khi ta cần thiết kế API quanh mutation trực tiếp và nil. Trước khi đến đó, hãy giữ model vừa xây: luôn hỏi value nào được copy, và value ấy có reference tới storage hay variable nào không.

Nếu câu trả lời là “không”, assignment thường tạo independence như int và array. Nếu câu trả lời là “có”, aliasing có thể là công cụ tốt hoặc một bug - tùy việc bạn có làm rõ ownership và mutation hay không.

Một quyết định ở ranh giới API

Khi một hàm nhận hoặc trả về slice, câu hỏi thiết kế không phải là “Go có copy không?” mà là: **ai được phép sửa storage này, và trong khoảng thời gian nào?** Một hàm lọc dữ liệu có thể trả về một view để tránh allocation; khi đó caller cần biết view ấy còn dùng chung buffer. Một hàm dựng response hoặc preview thường nên trả về dữ liệu độc lập, vì caller không có lý do để đoán một thay đổi sau đó sẽ đi ngược vào input.

Không có đáp án mặc định miễn phí. Copy tốn allocation và thời gian; chia sẻ tiết kiệm chúng nhưng làm contract về ownership quan trọng hơn. Trước khi tối ưu, hãy viết test cho hành vi mutation mà API hứa. Sau đó, khi đọc một bug khó hiểu, lần theo ba bước: value nào được gán, value đó reference storage nào, và ai còn có thể chạm storage ấy. Đó là cách biến “slice lạ quá” thành một cuộc điều tra có thể kiểm chứng.